

Name: Kshitiz Reddy.M

Reg No. A26MTAI0008

Dataset: Breast Cancer Wisconsin (Option 1, `load_breast_cancer()`) — binary classification of tumours as Malignant (0) or Benign (1).

Lab 4: Implementation of K-Nearest Neighbors (K-NN) and Random Forest Classifiers

Required Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, confusion_matrix, precision_score,
                             recall_score, f1_score, classification_report,
                             ConfusionMatrixDisplay)

# fix the seed so results can be reproduced
np.random.seed(42)

%matplotlib inline
plt.rcParams['figure.figsize'] = (6, 4)
```

Question 1: Dataset Loading and Exploration

Objective: Load the selected dataset and explore its structure before modelling.

1. Load the selected dataset (Breast Cancer, Wine, or Iris).
2. Display the first five samples.
3. Display the dataset dimensions.
4. Display the number of features.
5. Display the number of classes.
6. Check for missing values.

```
# 1. Load the Breast Cancer Wisconsin dataset
data = load_breast_cancer()
X = data.data          # feature matrix
y = data.target        # target: 0 = malignant, 1 = benign

# put it in a DataFrame so it's easy to look at
df = pd.DataFrame(X, columns=data.feature_names)
df['target'] = y

# 2. First five samples
df.head()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst texture	worst perimeter
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	17.33	184.60
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	23.41	158.60
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	25.53	152.60
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	26.50	98.60
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	16.67	152.60

5 rows × 31 columns

```
# 3. Dataset dimensions
print('Dataset shape (samples, features):', df.shape)

# 4. Number of features
print('Number of features:', len(data.feature_names))

# 5. Number of classes
print('Number of classes:', len(np.unique(y)))
print('Class names:', data.target_names)

# class distribution
print('\nClass distribution:')
print(pd.Series(y).value_counts().rename(index={0: 'malignant (0)', 1: 'benign (1)'}))
```

```
Dataset shape (samples, features): (569, 31)
Number of features: 30
Number of classes: 2
Class names: ['malignant' 'benign']
```

```
Class distribution:
benign (1)      357
malignant (0)   212
Name: count, dtype: int64
```

```
# 6. Check for missing values
print('Missing values (total):', df.isnull().sum().sum())

print('\n--- Data types ---')
df.info()

print('\n--- Statistical summary (first 5 features) ---')
df.describe().T.round(2).head()
```

```

Missing values (total): 0

--- Data types ---
<class 'pandas.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 31 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   mean radius                          569 non-null    float64
 1   mean texture                         569 non-null    float64
 2   mean perimeter                      569 non-null    float64
 3   mean area                          569 non-null    float64
 4   mean smoothness                    569 non-null    float64
 5   mean compactness                   569 non-null    float64
 6   mean concavity                     569 non-null    float64
 7   mean concave points                569 non-null    float64
 8   mean symmetry                      569 non-null    float64
 9   mean fractal dimension              569 non-null    float64
10   radius error                       569 non-null    float64
11   texture error                      569 non-null    float64
12   perimeter error                    569 non-null    float64
13   area error                        569 non-null    float64
14   smoothness error                   569 non-null    float64
15   compactness error                  569 non-null    float64
16   concavity error                    569 non-null    float64
17   concave points error               569 non-null    float64
18   symmetry error                     569 non-null    float64
19   fractal dimension error            569 non-null    float64
20   worst radius                      569 non-null    float64
21   worst texture                     569 non-null    float64
22   worst perimeter                   569 non-null    float64
23   worst area                        569 non-null    float64
24   worst smoothness                  569 non-null    float64
25   worst compactness                  569 non-null    float64
26   worst concavity                   569 non-null    float64
27   worst concave points              569 non-null    float64
28   worst symmetry                    569 non-null    float64
29   worst fractal dimension            569 non-null    float64
30   target                           569 non-null    int64
dtypes: float64(30), int64(1)
memory usage: 137.9 KB

--- Statistical summary (first 5 features) ---

```

	count	mean	std	min	25%	50%	75%	max
mean radius	569.0	14.13	3.52	6.98	11.70	13.37	15.78	28.11
mean texture	569.0	19.29	4.30	9.71	16.17	18.84	21.80	39.28
mean perimeter	569.0	91.97	24.30	43.79	75.17	86.24	104.10	188.50
mean area	569.0	654.89	351.91	143.50	420.30	551.10	782.70	2501.00
mean smoothness	569.0	0.10	0.01	0.05	0.09	0.10	0.11	0.16

Question 2: Data Preprocessing

Objective: Prepare the dataset for K-NN and Random Forest classification.

- Split the data into training and testing sets.
- Standardize the features (required for K-NN).

```

# 7. Split into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

print('Training samples:', X_train.shape[0])
print('Testing samples :', X_test.shape[0])
print('Class balance in train:', np.bincount(y_train))
print('Class balance in test :', np.bincount(y_test))

```

```

Training samples: 455
Testing samples : 114
Class balance in train: [170 285]
Class balance in test : [42 72]

```

```

# 8. Standardize the features (mean 0, std 1) -- required for K-NN
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # fit on the training set only
X_test_scaled = scaler.transform(X_test)      # reuse the same scaler on test

print('Train mean (first 5 features, should be ~0):', X_train_scaled.mean(axis=0)[:5].round(4))
print('Train std (first 5 features, should be ~1):', X_train_scaled.std(axis=0)[:5].round(4))

```

```
# NOTE: Random Forest is threshold/split based, not distance based, so it does
# NOT need scaling. I keep the unscaled X_train / X_test for Random Forest and
# use X_train_scaled / X_test_scaled only for K-NN below.
```

```
Train mean (first 5 features, should be ~0): [-0.  0. -0.  0.  0.]
Train std (first 5 features, should be ~1): [1.  1.  1.  1.  1.]
```

Question 3: K-NN Classification

Objective: Build, train, and evaluate a K-Nearest Neighbors classifier.

9. Create a K-NN classifier with an initial value of K = 5.
10. Train the classifier using the training data.
11. Predict the class labels for the test data.
12. Compute Accuracy, Confusion Matrix, Precision, Recall, and F1-score.

```
# 9. Create a K-NN classifier with K = 5
knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')

# 10. Train the classifier
knn.fit(X_train_scaled, y_train)

# 11. Predict the class labels for the test data
y_pred_knn = knn.predict(X_test_scaled)
```

```
# 12. Accuracy, Precision, Recall, F1-score
acc_knn = accuracy_score(y_test, y_pred_knn)
prec_knn = precision_score(y_test, y_pred_knn)
rec_knn = recall_score(y_test, y_pred_knn)
f1_knn = f1_score(y_test, y_pred_knn)

print(f'Accuracy : {acc_knn:.4f}')
print(f'Precision: {prec_knn:.4f}')
print(f'Recall : {rec_knn:.4f}')
print(f'F1-score : {f1_knn:.4f}')

print('\nClassification report:')
print(classification_report(y_test, y_pred_knn, target_names=data.target_names))
```

```
Accuracy : 0.9561
Precision: 0.9589
Recall : 0.9722
F1-score : 0.9655
```

```
Classification report:
              precision    recall  f1-score   support

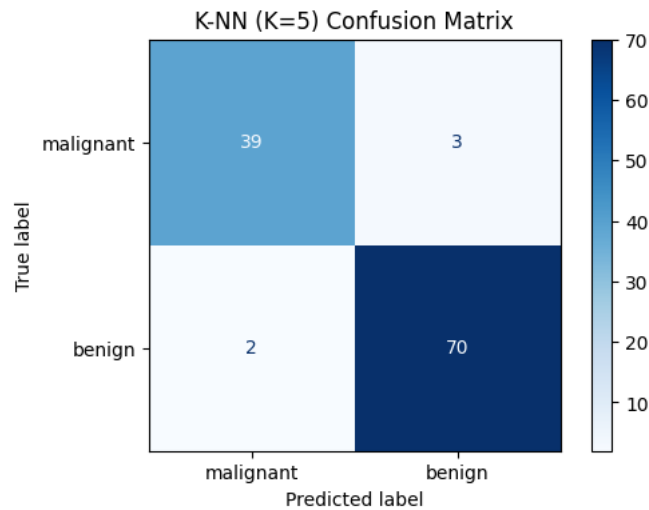
   malignant      0.95      0.93      0.94        42
    benign      0.96      0.97      0.97        72

   accuracy              0.96              114
  macro avg      0.96      0.95      0.95        114
 weighted avg      0.96      0.96      0.96        114
```

```
# Confusion matrix
cm_knn = confusion_matrix(y_test, y_pred_knn)
print('Confusion matrix (rows = actual, columns = predicted):')
print(cm_knn)

ConfusionMatrixDisplay(cm_knn, display_labels=data.target_names).plot(cmap='Blues')
plt.title('K-NN (K=5) Confusion Matrix')
plt.show()
```

Confusion matrix (rows = actual, columns = predicted):
[[39 3]
[2 70]]



Question 4: Random Forest Classification

Objective: Build, train, and evaluate a Random Forest classifier.

13. Build a Random Forest classifier.
14. Train the model using the training data.
15. Predict the test data.
16. Compute Accuracy, Confusion Matrix, and Classification Report.
17. Display the feature importance values and identify the top three most important features.

```
# 13. Build a Random Forest classifier
rf = RandomForestClassifier(n_estimators=100, random_state=42)

# 14. Train the model (note: uses the UNSCALED features -- RF doesn't need scaling)
rf.fit(X_train, y_train)

# 15. Predict the test data
y_pred_rf = rf.predict(X_test)
```

```
# 16. Accuracy, Confusion Matrix, Classification Report
acc_rf = accuracy_score(y_test, y_pred_rf)
cm_rf = confusion_matrix(y_test, y_pred_rf)

print(f'Accuracy: {acc_rf:.4f}')
print('\nConfusion matrix (rows = actual, columns = predicted):')
print(cm_rf)
print('\nClassification report:')
print(classification_report(y_test, y_pred_rf, target_names=data.target_names))
```

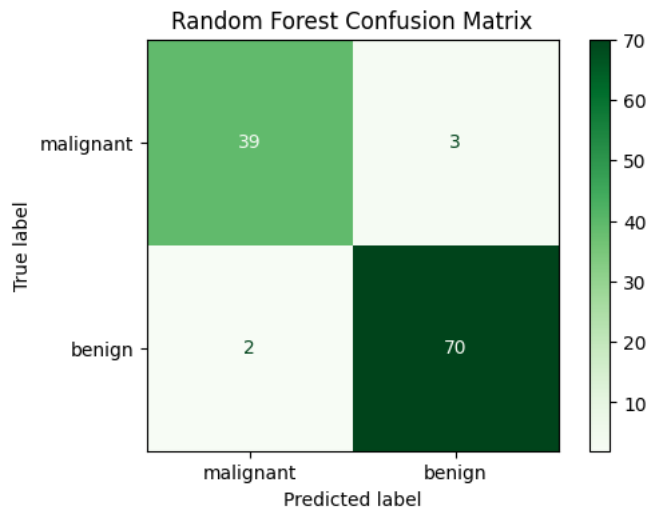
Accuracy: 0.9561

Confusion matrix (rows = actual, columns = predicted):
[[39 3]
[2 70]]

Classification report:

	precision	recall	f1-score	support
malignant	0.95	0.93	0.94	42
benign	0.96	0.97	0.97	72
accuracy			0.96	114
macro avg	0.96	0.95	0.95	114
weighted avg	0.96	0.96	0.96	114

```
ConfusionMatrixDisplay(cm_rf, display_labels=data.target_names).plot(cmap='Greens')
plt.title('Random Forest Confusion Matrix')
plt.show()
```



```
# 17. Feature importance -- top three most important features
importances = rf.feature_importances_
feat_imp_df = pd.DataFrame({
    'feature': data.feature_names,
    'importance': importances
}).sort_values(by='importance', ascending=False)

print('Top 3 most important features:')
print(feat_imp_df.head(3).to_string(index=False))

feat_imp_df.head(10)
```

```
Top 3 most important features:
      feature  importance
      worst area    0.140016
worst concave points 0.129530
      worst radius  0.097696
```

	feature	importance
23	worst area	0.140016
27	worst concave points	0.129530
20	worst radius	0.097696
7	mean concave points	0.090885
22	worst perimeter	0.072226
2	mean perimeter	0.069574
0	mean radius	0.068676
6	mean concavity	0.057638
3	mean area	0.049172
26	worst concavity	0.034340

Question 5 (Optional): K-NN from Scratch

Objective: Implement the core logic of K-NN using only NumPy, without Scikit-Learn.

18. Compute the Euclidean distance between a test sample and all training samples.
19. Sort the distances.
20. Select the K nearest neighbors.
21. Determine the predicted class using majority voting.

```
def euclidean_distance(a, b):
    # 18. distance between one test sample (b) and every training sample (a)
    return np.sqrt(np.sum((a - b) ** 2, axis=1))

def knn_predict_scratch(X_train, y_train, x_test, k=5):
    distances = euclidean_distance(X_train, x_test) # 18

    sorted_idx = np.argsort(distances) # 19. sort distances

    k_nearest_idx = sorted_idx[:k] # 20. K nearest neighbours
```

```

k_nearest_labels = y_train[k_nearest_idx]

counts = np.bincount(k_nearest_labels)          # 21. majority vote
return np.argmax(counts)

def knn_predict_all_scratch(X_train, y_train, X_test, k=5):
    return np.array([knn_predict_scratch(X_train, y_train, x, k) for x in X_test])

# Sanity check: run the from-scratch version on the same (scaled) test set
y_pred_scratch = knn_predict_all_scratch(X_train_scaled, y_train, X_test_scaled, k=5)

acc_scratch = accuracy_score(y_test, y_pred_scratch)
print(f'From-scratch K-NN (K=5) Accuracy : {acc_scratch:.4f}')
print(f'Scikit-learn K-NN (K=5) Accuracy : {acc_knn:.4f}')
print('Predictions identical to sklearn :', np.array_equal(y_pred_scratch, y_pred_knn))

From-scratch K-NN (K=5) Accuracy : 0.9561
Scikit-learn K-NN (K=5) Accuracy : 0.9561
Predictions identical to sklearn : True

```

Question 6: Effect of K in K-NN

Objective: Study how the choice of K affects K-NN performance.

22. Train K-NN models using K = 3, 5, 7, and 9.
23. Complete the table below with the test accuracy for each value of K.
24. Identify which value of K gives the best performance and explain your observations.

```

k_values = [3, 5, 7, 9]
k_accuracies = []

for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k, metric='euclidean')
    model.fit(X_train_scaled, y_train)
    preds = model.predict(X_test_scaled)
    acc = accuracy_score(y_test, preds)
    k_accuracies.append(acc)
    print(f'K = {k} -> Test Accuracy = {acc:.4f}')

k_results_df = pd.DataFrame({'K': k_values, 'Test Accuracy': k_accuracies}).set_index('K')
k_results_df.round(4)

```

```

K = 3 -> Test Accuracy = 0.9825
K = 5 -> Test Accuracy = 0.9561
K = 7 -> Test Accuracy = 0.9737
K = 9 -> Test Accuracy = 0.9737

```

Test Accuracy

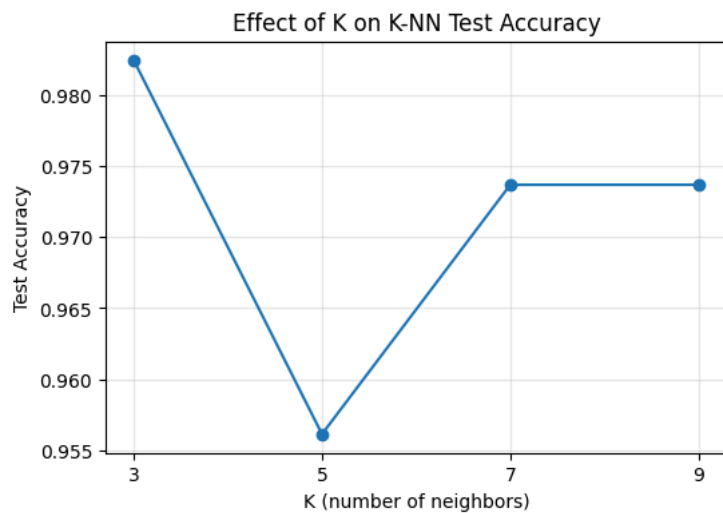
K	
3	0.9825
5	0.9561
7	0.9737
9	0.9737

```

plt.plot(k_values, k_accuracies, marker='o', color='tab:blue')
plt.xlabel('K (number of neighbors)')
plt.ylabel('Test Accuracy')
plt.title('Effect of K on K-NN Test Accuracy')
plt.xticks(k_values)
plt.grid(True, alpha=0.3)
plt.show()

best_k = k_values[int(np.argmax(k_accuracies))]
print(f'Best performing K in this run: {best_k} (Accuracy = {max(k_accuracies):.4f})')

```



Best performing K in this run: 3 (Accuracy = 0.9825)

Question 7: Effect of Number of Trees in Random Forest

Objective: Study how the number of trees affects Random Forest performance.

25. Train Random Forest models using 10, 50, 100, and 200 trees.
26. Complete the table below with the test accuracy for each setting.
27. Discuss how increasing the number of trees affects model performance and computational cost.

```
n_estimators_list = [10, 50, 100, 200]
rf_accuracies = []
rf_train_times = []

for n in n_estimators_list:
    model = RandomForestClassifier(n_estimators=n, random_state=42)

    start = time.time()
    model.fit(X_train, y_train)
    train_time = time.time() - start

    preds = model.predict(X_test)
    acc = accuracy_score(y_test, preds)

    rf_accuracies.append(acc)
    rf_train_times.append(train_time)
    print(f'Trees = {n:>3} -> Test Accuracy = {acc:.4f}, Training Time = {train_time:.4f}s')

rf_results_df = pd.DataFrame({
    'Number of Trees': n_estimators_list,
    'Test Accuracy': rf_accuracies,
    'Training Time (s)': rf_train_times
}).set_index('Number of Trees')
rf_results_df.round(4)
```

```
Trees = 10 -> Test Accuracy = 0.9386, Training Time = 0.0175s
Trees = 50 -> Test Accuracy = 0.9561, Training Time = 0.0824s
Trees = 100 -> Test Accuracy = 0.9561, Training Time = 0.1648s
Trees = 200 -> Test Accuracy = 0.9561, Training Time = 0.3275s
```

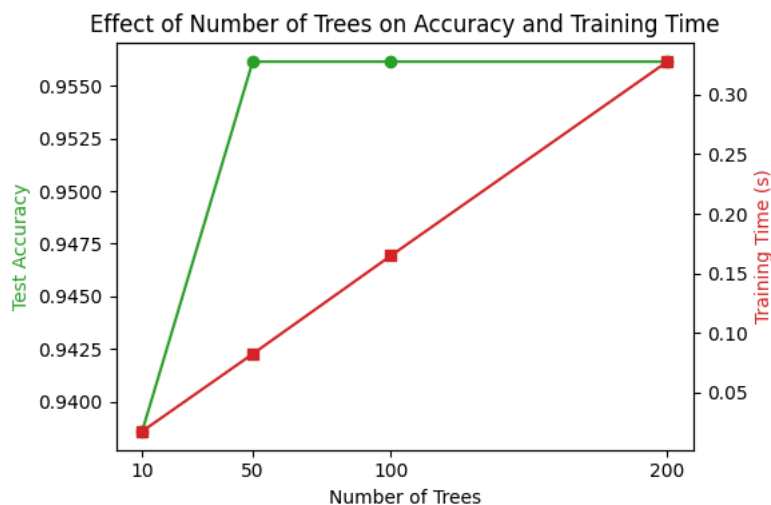
	Test Accuracy	Training Time (s)
Number of Trees		
10	0.9386	0.0175
50	0.9561	0.0824
100	0.9561	0.1648
200	0.9561	0.3275

```
fig, ax1 = plt.subplots()
ax1.plot(n_estimators_list, rf_accuracies, marker='o', color='tab:green', label='Accuracy')
ax1.set_xlabel('Number of Trees')
ax1.set_ylabel('Test Accuracy', color='tab:green')
ax1.set_xticks(n_estimators_list)

ax2 = ax1.twinx()
```

```
ax2.plot(n_estimators_list, rf_train_times, marker='s', color='tab:red', label='Training Time')
ax2.set_ylabel('Training Time (s)', color='tab:red')

plt.title('Effect of Number of Trees on Accuracy and Training Time')
fig.tight_layout()
plt.show()
```



Question 8: Comparing K-NN and Random Forest

Objective: Compare the K-NN and Random Forest classifiers on the chosen dataset.

28. Record the Training Time, Prediction Time, Accuracy, Interpretability, and Need for Feature Scaling for each model in the table below.
29. Based on your results, determine which algorithm is more suitable for the chosen dataset and explain why.

```
# --- K-NN timing ---
knn_final = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
start = time.time(); knn_final.fit(X_train_scaled, y_train); knn_train_time = time.time() - start
start = time.time(); knn_preds_final = knn_final.predict(X_test_scaled); knn_pred_time = time.time() - start
knn_acc_final = accuracy_score(y_test, knn_preds_final)

# --- Random Forest timing ---
rf_final = RandomForestClassifier(n_estimators=100, random_state=42)
start = time.time(); rf_final.fit(X_train, y_train); rf_train_time = time.time() - start
start = time.time(); rf_preds_final = rf_final.predict(X_test); rf_pred_time = time.time() - start
rf_acc_final = accuracy_score(y_test, rf_preds_final)

comparison_df = pd.DataFrame({
    'Parameter': ['Training Time (s)', 'Prediction Time (s)', 'Accuracy',
                  'Interpretability', 'Need for Feature Scaling'],
    'K-NN': [f'{knn_train_time:.5f}', f'{knn_pred_time:.5f}', f'{knn_acc_final:.4f}',
             'Low (no explicit model, just stored data + distances)', 'Yes (required)'],
    'Random Forest': [f'{rf_train_time:.5f}', f'{rf_pred_time:.5f}', f'{rf_acc_final:.4f}',
                     'Medium-High (feature_importances_, individual trees inspectable)', 'No (not required)']
})
comparison_df
```

	Parameter	K-NN	Random Forest
0	Training Time (s)	0.00211	0.15687
1	Prediction Time (s)	0.00172	0.00598
2	Accuracy	0.9561	0.9561
3	Interpretability	Low (no explicit model, just stored data + dis...	Medium-High (feature_importances_, individual ...
4	Need for Feature Scaling	Yes (required)	No (not required)

v

```
# 30. Top three most important features (from the rf model trained in Question 4)
top3 = feat_imp_df.head(3)
print('Top 3 most important features:')
print(top3.to_string(index=False))
```

31. Explain how feature importance can assist in feature selection and model interpretation.

	feature	importance
Top 3 most important features:	worst area	0.140016
	worst concave points	0.129530
	worst radius	0.097696

```
# Feature importance bar chart
plt.figure(figsize=(8, 8))
top_n = feat_imp_df.head(10)
plt.barh(top_n['feature'][::-1], top_n['importance'][::-1], color='tab:green')
plt.xlabel('Importance')
```